SARVAJANIK COLLEGE OF ENGINEERING AND TECHNOLOGY

COMPUTER ENGINEERING DEPARTMENT

B. Tech III, SEM V, Div- II

(ODD 2026-27)

Open Ended Problem (OEP)

on

"CivicShield: Privacy-Preserving Civic Incident Intelligence Platform"

Subject Name: Web Technologies (BTCO13502)

Prepared and Presented by Group No. : 7

Nirav Panwala : ET24BTCO181

Mevada Drashtant Pareshkumar : ET24BTCO176

Priyanshu Makani : ET24BTCO170

Merit Patel : ET24BTCO185

Heet Sutariya : ET24BTCO204

Outline

1. Introduction
2. Modules with details
3. Database Design
4. References

1. Introduction

1.1 Background and Problem Statement

Municipal bodies receive civic complaints such as potholes, water leakage, broken streetlights and garbage overflow through phone calls, paper forms and scattered messaging groups. Three problems follow. The same issue is reported many times and is handled as many separate cases. Urgent issues are not separated from minor ones. Citizens are asked for names, phone numbers and exact locations, and many avoid reporting because they fear this information will be exposed or misused.

CivicShield is a role-based web application that addresses these problems. Citizens report local incidents and track them later without needing to reveal who they are. Administrators merge related reports into one case, rank cases by urgency and assign them to field officers. Officers submit proof of work, which an administrator verifies before the case is closed. Throughout, the system limits how much reporter identity and precise location is exposed.

1.2 Objectives

- Provide a citizen portal for reporting incidents and tracking their status with a private token.
- Separate individual citizen reports from consolidated incidents, so many reports can feed one municipal case.
- Detect likely duplicate reports automatically but keep the final decision with an administrator.
- Rank incidents with a transparent, explainable priority score.
- Support the full workflow of assignment, field resolution, evidence upload, verification and closure.
- Show analytics and a map to administrators using obfuscated coordinates.
- Apply defensive web practices for authentication, forms, database access and file uploads.

1.3 User Roles

Role	Main capabilities
Citizen / guest	Submit a report (anonymous or pseudonymous), attach an image, optionally leave contact details, track status with a token.
Field officer	See only assigned incidents, acknowledge work, add field notes, submit resolution evidence (photo and inspection location).
Administrator	Review duplicates, manage the incident queue, assign officers, verify or reject resolutions, view analytics and audit data.

1.4 Technology Stack

Layer	Technology
Server side	Native PHP 8.x with a small set of service classes (no framework)
Database	MySQL 8 / MariaDB (InnoDB, utf8mb4) accessed through PDO prepared statements
Front end	HTML5, CSS3, Bootstrap 5.3, JavaScript (ES6+), jQuery, Fetch/AJAX
Maps	Leaflet.js with OpenStreetMap tiles
Environment and testing	XAMPP (Apache + MySQL) locally; PHP command-line regression scripts

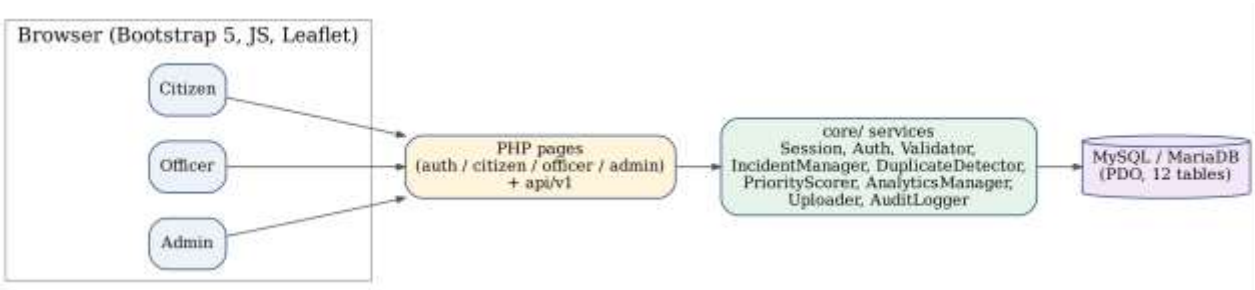


Figure 1: High-level architecture of CivicShield

1.5 End-to-End Workflow

1. A citizen files a report with category, severity, description and location; the system returns a private tracking token.
2. The duplicate detector scores the report against nearby open incidents and queues likely matches.
3. An administrator confirms a duplicate (report joins an existing incident) or marks it distinct (a new incident is created).
4. The priority score is calculated, and the incident appears in the admin queue.
5. The administrator assigns the incident to a field officer with instructions.
6. The officer starts work and submits a completion photo, notes and inspection location.
7. The administrator verifies the evidence, and the incident moves to resolved and then closed.
8. At every stage the citizen can check the public status using the token, without any personal data being shown.

Scope note. CivicShield is an academic prototype. Its privacy measures reduce exposure in application views and public endpoints. They do not give absolute anonymity, and coordinate fuzzing is not a formal differential-privacy guarantee.

1.6 Known Limitations

- Coordinate fuzzing limits precision but cannot prevent every re-identification attempt, and it is not formal differential privacy.
- The duplicate and priority scores are heuristics; a high duplicate score is evidence for review, not proof, and safety-critical reports must be read even when their score is low.
- GPS inaccuracy and very short descriptions reduce the quality of duplicate matching.
- Server logs, database backups and privileged misuse are outside the application's protections; real deployment needs HTTPS, restricted database permissions and an independent security review.
- The system has been tested with CLI regression scripts and demo data only, not with real citizen traffic.

2. Modules with Details

The application is organised into role-based page folders (auth/, citizen/, officer/, admin/, api/v1/) that call shared service classes in core/. The modules below describe what each part does and how it works internally.

2.1 Authentication, Sessions and Access Control

Users register and log in through the auth pages. Passwords are stored only as hashes (bcrypt/Argon2id), and each account has a role (citizen, officer or admin) and a status (active, suspended or pending). Every protected page calls a guard such as requireAuth('admin') or requireAuth('officer'), so authorisation is enforced on the server and not merely by hiding menu items. The Session class configures secure cookie options and regenerates the session ID on login. Failed logins are recorded in the login_rate_limits table, so throttling survives restarts and cannot be avoided by clearing cookies. Redirect targets after login are checked by Session::sanitizeRedirect() and limited to safe local paths.

2.2 Citizen Reporting Module

The report form collects a category, title, description, severity, a map location and an optional photo. A citizen can submit as pseudonymous (logged in) or anonymous (guest). Phone or email is optional and is kept in a separate table with an explicit consent flag. Every submission passes CSRF validation and server-side checks in the Validator class. The Uploader accepts only allowed image types within a 5 MB limit, checks the file with getimagesize, and stores it under a generated name. A SHA-256 digest is saved for integrity. After submission, the citizen receives a random tracking token once; the database keeps only its SHA-256 hash. The client IP and user agent are stored only as salted hashes, for abuse control.

2.3 Public Tracking Module

A citizen enters the token on track.php or sends it to the versioned endpoint api/v1/track.php. The server hashes the token and looks up the matching report. The endpoint accepts POST only, so the token never appears in a URL, server log or browser history. It returns lifecycle status and public status notes only. It never returns contact details, raw coordinates, internal staff notes or token hashes, and it sends no-store and no-referrer headers. A separate api/v1/health.php returns minimal status and hides database error details.

2.4 Duplicate Detection Module

When a report arrives, DuplicateDetector compares it with nearby open incidents within a configurable radius (default 50 m). It gives each candidate a composite score from four signals:

Signal	Weight	How it is computed
Spatial proximity	40%	Haversine distance; score falls linearly from 100 at 0 m to 0 at the radius
Category match	30%	100 if categories are equal, otherwise 0
Text similarity	20%	60% token Jaccard + 40% character-level similarity of title and description
Time proximity	10%	100 within 2 hours, decaying to 0 after 14 days

Score = 0.40 × spatial + 0.30 × category + 0.20 × text + 0.10 × time. Candidates at or above the threshold (50 by default) are queued in incident_duplicate_candidates. An administrator then confirms the duplicate, which links the report to the existing incident, or marks it distinct, which promotes it to a new incident. Location alone never merges reports, because GPS error and short descriptions can mislead the score.

2.5 Priority Scoring Module

PriorityScorer gives each incident a 0–100 score that helps administrators decide what to review first. It is a fixed rule-based formula, not machine learning. Each factor is scaled to 0–100 and then weighted:

Factor	Weight	Scoring rule
Submitted severity	35%	low 25, medium 50, high 75, critical 100
Distinct report count	25%	1 report = 20, 2 = 45, 3 = 70, 4 = 85, 5 or more rises to 100
Elapsed time open	20%	up to 6 h = 20; 24 h = 40; 48 h = 65; 72 h = 85; beyond 72 h = 100
Category urgency (SLA)	20%	SLA ≤ 12 h = 100; ≤ 24 h = 75; ≤ 36 h = 50; longer = 35

The result is rounded and clamped to 0–100, and the breakdown is stored as JSON in the incident so the ranking can be explained. The score is decision support only; it does not dispatch or close anything automatically.

2.6 Incident Lifecycle and Admin Queue

The administrator queue lists incidents with filters (status, category, minimum priority, date range, search) and pagination. Every status change goes through IncidentManager, which accepts only transitions listed in an allowed-transitions table. Illegal jumps, such as reported directly to closed, are rejected. Every accepted change is written to incident_status_history together with its reason.

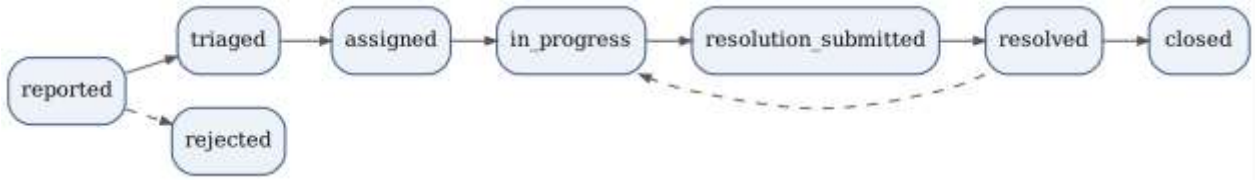


Figure 2: Main incident lifecycle (dashed arrows are reopen / reject paths)

2.7 Assignment and Officer Module

From the incident detail page, an administrator assigns an officer, usually from the matching department, with written instructions. The officer sees only his or her own assignments and may perform just two transitions: assigned → in_progress and in_progress → resolution_submitted. To submit, the officer must upload a completion photo, notes and on-site inspection coordinates, which are stored in resolution_evidence. The incident then waits in resolution_submitted. An administrator can verify the evidence, which moves the incident to resolved and later closed, or reject it or request rework.

2.8 Analytics and Privacy-Preserving Map

AnalyticsManager gives administrators KPIs and charts: incidents by status, category and priority, and resolution trends. The map module (Leaflet) plots incidents using the obfuscated_lat and obfuscated_lng columns, which are perturbed within a configured radius (150 m by default), not the raw report coordinates. Raw coordinates stay in restricted database records and are never included in map payloads.

2.9 Audit Logging, Security Controls and Testing

AuditLogger records administrative and security actions (for example assignment, duplicate decisions and evidence verification) with the actor, target and a hashed IP address, while keeping citizen personal data out of the log. Other controls are CSRF tokens on every state-changing form, PDO prepared statements against SQL injection, output escaping, and an .htaccess rule in uploads/ that blocks script execution. Nine PHP command-line test scripts in tests/ check the main workflows: duplicate detection, queue and assignment, analytics, public tracking, login rate limiting, resolution evidence and distinct-report promotion. Passing these tests is not a penetration test, and the application should receive an independent security review before it handles real citizen data.

2.10 Application Endpoints

Endpoint / page	Method	Purpose
auth/login.php, register.php, logout.php	GET / POST	Authentication and account creation
citizen/report.php	GET / POST	Report form and submission (CSRF protected)
track.php	GET / POST	Token-based status lookup page
api/v1/track.php	POST only	Public lifecycle status for a token (GET returns 405)
api/v1/health.php	GET	Minimal service health status
admin/incidents.php, duplicates.php, analytics.php	GET / POST	Queue, duplicate review, analytics
officer/incidents.php, incident_detail.php	GET / POST	Assigned work, field updates, evidence upload

3. Database Design

The schema (civicshield_db) has 12 InnoDB tables using utf8mb4. All relationships are enforced by foreign keys, and the columns used most in filters, such as status, severity, priority_score, officer_id and the coordinate pairs, are indexed. The central design decision is to keep **incident_reports** (what individual citizens submitted) separate from **incidents** (the consolidated case that the municipality works on). Many reports can therefore point to one incident, and sensitive intake data stays away from the operational and public views.

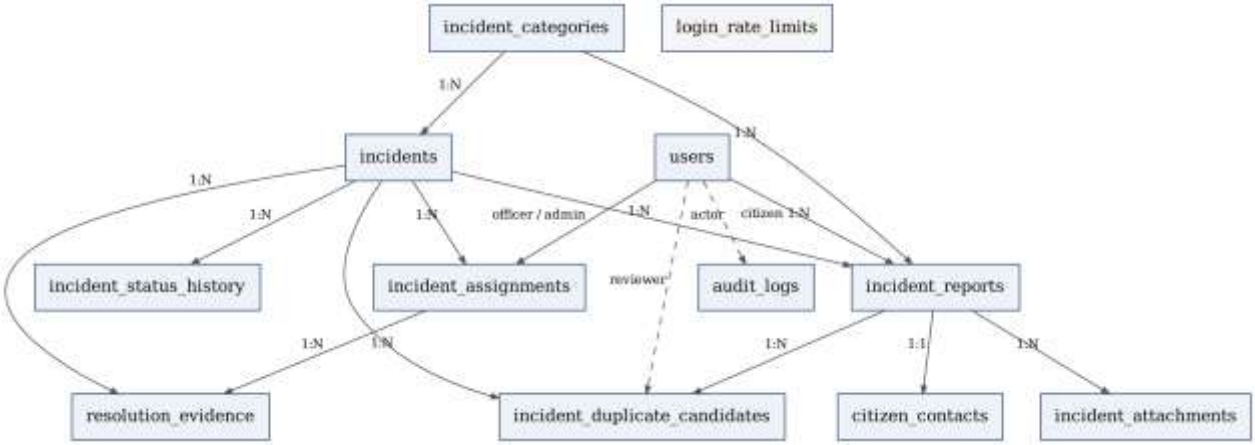


Figure 3: Entity-relationship overview of the 12 tables

3.1 Table Summary

Table	Purpose and key columns
users	Accounts. username, email (optional for citizens), password_hash, role (citizen/officer/admin), department, status.
incident_categories	Category, owning department_code, default_severity, resolution_sla_hours, is_active.
incidents	Consolidated case. incident_reference, category_id, severity, priority_score, priority_breakdown (JSON), status, obfuscated_lat/lng, fuzz_radius_meters, neighborhood, reports_count, resolved_at, closed_at.
incident_reports	Individual submissions. report_reference, tracking_token_hash (SHA-256), incident_id (nullable), citizen_id (nullable), submitted_severity, submission_mode, raw_lat/lng (restricted), ip_hash, user_agent_hash, status.
citizen_contacts	Optional name, phone, email and consent flag; one-to-one with a report and deleted with it.
incident_attachments	Uploaded image metadata: stored name, MIME type, size, sha256_hash, exif_scrubbed flag.
incident_duplicate_candidates	report_id, candidate_incident_id, match_score, distance, time difference, text similarity, review_status, reviewer, decision_notes. Unique on (report, candidate).
incident_assignments	incident_id, officer_id, assigned_by_user_id, assignment_status, instructions, assigned/acknowledged/completed times.
resolution_evidence	Officer's completion photo (with SHA-256), notes, inspection_lat/lng, verification_status, verifier and notes.
incident_status_history	Each transition: status_from, status_to, reason, changed_by, is_public flag.
audit_logs	actor, action, target_type/target_id, ip_hash, metadata_json (no citizen personal data).
login_rate_limits	rate_key hash, ip_address, username, attempted_at, for persistent login throttling.

3.2 Key Relationships

- **incidents → incident_reports (1:N):** a consolidated incident is linked to every report that describes it; the link is empty while a report is still pending review.
- **incident_reports → incident_duplicate_candidates ← incidents:** each row records one proposed match between a report and an existing incident, with the score components and the administrator's decision.
- **incidents → incident_assignments → resolution_evidence:** an incident can have several assignments over time (for example after reassignment), and each assignment can produce evidence that is later verified.
- **users** appear as citizen (report owner), officer (assignee), administrator (assigner, reviewer, verifier) and actor in audit_logs; login_rate_limits is standalone.

3.3 Design Decisions

- **Privacy by separation:** contact details live in their own table, and raw coordinates are only in incident_reports and resolution_evidence. Public and map queries read the obfuscated incident columns only.
- **Hashed secrets:** tracking tokens, IP addresses and user agents are stored as SHA-256 hashes, so a database leak does not reveal them directly.
- **Referential rules:** deleting a citizen account sets report ownership to NULL (the report remains); deleting a report removes its contact and attachments; categories and officers cannot be deleted while incidents or assignments depend on them (RESTRICT).
- **Traceability:** status history and audit logs record who did what and when, which supports accountability without exposing personal data.
- **Controlled values:** ENUM columns for role, status, severity and review state prevent invalid values at the database level.

4. References

[1] PHP Manual, “PDO – PHP Data Objects” and “Sessions”. https://www.php.net/manual/

[2] MySQL 8.0 Reference Manual, “InnoDB and FOREIGN KEY Constraints”. https://dev.mysql.com/doc/refman/8.0/en/

[3] Bootstrap 5.3 Documentation. https://getbootstrap.com/docs/5.3/

[4] Leaflet 1.9 Documentation. https://leafletjs.com/reference.html

[5] OpenStreetMap Foundation. https://www.openstreetmap.org/

[6] OWASP Cheat Sheet Series: CSRF Prevention, Password Storage, SQL Injection Prevention and File Upload. https://cheatsheetseries.owasp.org/

[7] R. W. Sinnott, “Virtues of the Haversine”, Sky and Telescope, vol. 68, no. 2, 1984 (great-circle distance formula).

[8] Project source repository: CivicShield – Privacy-Preserving Civic Incident Intelligence Platform (README and docs/ folder).